Distributed Performance Testing Platform

Project Proposal for UCS503P
Submitted to: Dr. Paramveer Kaur
Thapar Institute of Engineering and Technology

Jatin Bajaj - `1024170389`
Krish Sahni - `1024170390`
Rishabh Dutt - `10240170393`

August 16, 2026

1 Higher-order goal

. . . primary objective

This project aims to build a scalable, backend-focused distributed performance testing platform capable of generating and analyzing high volumes of concurrent HTTP traffic against APIs and web applications, including applications running locally on a developer's machine. It focuses primarily on distributed backend architecture, scalable remote request generation, secure connectivity to local applications, real-time telemetry processing, and automated performance analysis.

The objective is not merely to generate load and display graphs, but to provide an engineering platform capable of determining system capacity, identifying performance bottlenecks, detecting breaking points, comparing performance across different application versions, and enabling users with limited hardware to use remote cloud-based load generators for testing locally hosted applications.

2 Time-to-value

. . . primary heuristic

- The initial iteration will focus on implementing the core controller-worker architecture and generating concurrent HTTP traffic against configurable API endpoints.
- Subsequent iterations will introduce distributed cloud workers, a lightweight local load-testing agent, secure remote connectivity, telemetry streaming, metric aggregation, real-time dashboards, adaptive stress testing, and performance regression testing.
- Development will prioritize measurable backend functionality over visual complexity, with each iteration producing a testable performance-testing capability.

3 Problem Statement

Existing performance-testing tools can generate large volumes of requests, but a developer running a local web server on a low-resource machine may not have sufficient CPU, memory, or network capacity to generate large workloads from the same machine. In addition, remotely generated traffic cannot directly access a developer's `localhost` without a secure communication mechanism.

The proposed platform addresses the following challenges:

- **Limited scalability:** A single load generator may become a bottleneck when thousands of concurrent requests are required.
- **Distributed load generation:** Large workloads need to be distributed across multiple cloud machines or worker processes so that the load-generation infrastructure does not become the bottleneck.
- **Limited client hardware:** Developers and students may have low-resource machines that cannot efficiently generate thousands of concurrent requests while also running the application under test.
- **Local application accessibility:** Cloud-based workers cannot directly access a developer's `localhost`; a lightweight local agent and secure outbound communication channel are required to route remote test traffic to the local application.
- **Limited real-time analysis:** Raw request results need to be aggregated into useful metrics such as throughput, latency percentiles, and error rates.
- **Manual breaking-point detection:** Developers often need to manually increase traffic and inspect results to determine when a system becomes unstable.
- **Performance regressions:** Performance degradation between application versions may not be immediately visible without systematic comparison of historical test results.

4 Proposed Solution

. . . software proposition

4.1 Overview

Build a backend-focused distributed load-testing platform consisting of a central controller, multiple cloud-based load-generator workers, a lightweight local load-testing agent, a secure communication gateway, a metrics aggregation pipeline, persistent test storage, and a real-time monitoring and visualization interface.

The local agent enables the platform to test applications that are running on a developer's `localhost`. The agent establishes a secure outbound connection to the platform and forwards test traffic from remote workers to the locally hosted application. This allows the load-generation workload to be executed remotely while the developer's machine primarily runs the application under test.

The platform can therefore allow a developer with a low-resource machine to run their application locally while using cloud-based workers to generate thousands of concurrent requests. The collected telemetry will be processed and presented through real-time graphs and visualizations, allowing users to observe application performance while the test is running.

The platform will provide:

- **Central controller:** Creates, configures, schedules, starts, and stops performance tests.
- **Load-generator workers:** Cloud-based, horizontally scalable workers that generate HTTP traffic concurrently and execute assigned workloads.
- **Local load-testing agent:** A lightweight process running on the user's machine that securely connects to the platform and forwards remote test traffic to applications running on `localhost`.
- **Communication gateway:** Maintains secure connections between cloud workers and local agents without exposing the developer's local application directly to the public Internet.

- **Metrics aggregation pipeline:** Processes high-volume telemetry generated by workers and calculates performance metrics such as throughput, requests per second, latency percentiles, error rates, timeouts, response codes, and concurrency.
- **Historical storage:** Stores test configurations, execution results, aggregated metrics, and performance history.
- **Real-time monitoring and visualization:** Displays live performance graphs for metrics such as requests per second, throughput, P50/P95/P99 latency, error rate, concurrency, response-code distribution, and SLO violations during test execution.

4.2 Core workflow

1. The user creates a performance test by configuring the target endpoint, virtual users, request rate, ramp-up period, duration, authentication, headers, payloads, and scenarios.
2. If the target application is locally hosted, the user starts the lightweight local load-testing agent and registers the local target endpoint, such as `localhost:8080`.
3. The local agent establishes a secure outbound connection to the platform so that remote workers can reach the local application through the controlled communication channel.
4. The central controller validates the configuration and distributes the workload across available cloud load-generator workers.
5. Workers generate concurrent HTTP requests and route them through the communication layer to the local agent when the target is locally hosted, or directly to a remotely accessible target when appropriate.
6. Workers collect request-level telemetry and asynchronously stream it to the metrics aggregation pipeline.
7. The aggregation layer calculates throughput, requests per second, latency percentiles, error rate, timeouts, response codes, and concurrent requests.
8. Aggregated metrics are persisted and simultaneously streamed to the dashboard for real-time monitoring and visualization.
9. After execution, the platform evaluates configured SLOs and determines whether the application remained within acceptable performance limits.

5 Performance Metrics

. . . measurable output

The platform will calculate and expose the following metrics:

- **Throughput:** Total number of successfully processed requests during a test.
- **Requests per second (RPS):** Number of requests generated or completed per second.
- **Latency percentiles:** P50, P95, and P99 latency values representing typical, high, and worst-case response behavior.
- **Error rate:** Percentage of requests resulting in failures.
- **Timeouts:** Number and percentage of requests that exceeded the configured timeout.
- **Response codes:** Distribution of HTTP status codes such as 2xx, 4xx, and 5xx responses.
- **Concurrency:** Number of active requests or virtual users during different stages of the test.

6 Adaptive Stress Testing *... capacity discovery*

Instead of using only a fixed load, the platform will support adaptive stress testing.

The system will progressively increase traffic according to a configured strategy and continuously evaluate application performance against predefined Service Level Objectives (SLOs).

Example SLOs include:

- P95 latency must remain below a configured threshold.
- P99 latency must remain below a configured threshold.
- Error rate must remain below a configured percentage.
- Request timeout rate must remain below a configured limit.
- Minimum throughput must remain above a configured RPS threshold.

The **maximum sustainable throughput** will be identified as the highest load at which the application continues to satisfy the configured SLOs.

The **breaking point** will be identified when increasing load causes the application to violate one or more critical SLOs, such as a significant increase in latency, error rate, or timeouts.

The system can progressively narrow the load range around the first SLO violation to estimate the capacity boundary more precisely.

This allows the system to automatically determine the approximate capacity boundary of the tested application rather than requiring manual inspection.

7 Performance Regression Testing *... version comparison*

The platform will support performance regression testing by storing historical test results and associating them with application versions or builds.

A new performance test can be compared against a previous baseline to identify degradation in:

- P50/P95/P99 latency.
- Throughput and requests per second.
- Error and timeout rates.
- Maximum sustainable throughput.
- Resource or concurrency behavior when such measurements are available.

A performance regression occurs when a newer application version performs significantly worse than the established baseline under comparable test conditions.

This functionality also makes the platform suitable for integration into a CI/CD pipeline, where performance tests can be executed automatically for application builds.

8 System Architecture *... backend engineering focus*

The proposed platform will follow a distributed architecture in which load generation is separated from the system under test. Cloud-based workers perform the computationally intensive request generation, while a lightweight local agent provides controlled access to applications running on a developer's `localhost`.

The high-level architecture is:

`Controller` → `Cloud Workers` → `Secure Gateway` → `Local Agent` → `localhost Application`

For remotely accessible applications, workers may communicate with the target directly without using the local agent.

- **Controller:** Maintains test configuration, manages test lifecycle, schedules workloads, and coordinates workers.
- **Load Generators:** Horizontally scalable cloud workers that generate concurrent HTTP traffic without placing the primary load-generation cost on the user's local machine.
- **Local Agent:** A lightweight client running beside a locally hosted application and maintaining a secure outbound connection to the platform.
- **Secure Communication Gateway:** Routes requests between cloud workers and registered local agents while controlling authentication, authorization, connection state, and request routing.
- **Telemetry Pipeline:** Receives high-volume execution data asynchronously from workers.
- **Metrics Aggregator:** Converts raw telemetry into aggregated performance metrics and percentile statistics.
- **Persistent Storage:** Stores test definitions, execution metadata, historical metrics, and regression baselines.
- **WebSocket Layer:** Provides real-time performance updates to connected dashboard clients.

The architecture is designed so that additional cloud load-generator workers can be added horizontally when higher traffic generation capacity is required. This separates the capacity of the load-generation infrastructure from the hardware available to the developer running the application under test.

A developer with a low-resource machine can therefore run a local application and use multiple remote workers to generate a large concurrent workload. The local agent performs request forwarding and communication management rather than acting as the primary load generator.

9 Solution Approach *... engineering focus*

The project will focus on backend engineering challenges rather than AI/ML-based prediction.

9.1 Concurrent Request Generation

Workers will use asynchronous or non-blocking networking techniques to efficiently generate a large number of concurrent HTTP requests without requiring one dedicated blocking thread for every request.

9.2 Distributed Coordination

The controller will divide workload among cloud workers and maintain the state of active performance tests. Workers will report their execution status and telemetry back to the controller or telemetry pipeline. Worker registration, health checks, heartbeats, and workload assignment will allow the controller to manage a changing set of load-generation nodes.

9.3 Local Application Connectivity

A lightweight local agent will maintain a secure outbound connection to the platform. When a test targets `localhost`, the platform will route requests through this connection to the locally running application.

The design avoids requiring the user's application to be directly exposed to the public Internet and prevents the local machine from being responsible for generating the complete workload.

9.4 Rate Limiting

Request generation will support configurable request rates to prevent uncontrolled traffic generation and to implement controlled load profiles.

9.5 Telemetry Streaming

Workers will stream high-volume execution data rather than waiting until the complete test finishes. This enables real-time aggregation and monitoring.

9.6 Fault Tolerance

The platform will account for worker failures and communication issues so that individual worker failures do not unnecessarily terminate the complete testing infrastructure.

10 Real-time Monitoring

. . . live system visibility

During a running test, aggregated metrics will be transmitted to the dashboard through WebSockets.

The dashboard will provide real-time visibility into:

- Current requests per second.
- Active concurrent requests.
- P50, P95, and P99 latency.
- Error and timeout rates.
- HTTP response-code distribution.
- Current load level during adaptive stress testing.
- SLO violations and detected performance limits.

The visualization layer will present these metrics through real-time graphs and charts, allowing users to observe changes in latency, throughput, concurrency, error rates, and load levels during test execution.

This avoids the need to wait until the test finishes before understanding the application's current behavior.

11 CI/CD Integration

. . . automated regression detection

The platform can be integrated into CI/CD pipelines so that performance tests become an automated quality gate.

A typical pipeline can execute the following workflow:

1. Build a new application version.
2. Deploy the application to a test environment.
3. Trigger the performance-testing platform through an API.
4. Execute a predefined workload.
5. Collect and aggregate performance metrics.
6. Compare results against the stored baseline.
7. Mark the pipeline as successful or failed according to configured performance SLOs.

This enables performance degradation to be detected before a build is released to production.

12 Evaluation Criteria

. . . measurable and attributable

The project will be evaluated using measurable technical metrics.

12.1 Primary Evaluation Metrics

- **Load generation capacity:** Maximum requests per second generated by the platform.
- **Concurrency:** Maximum number of concurrent virtual users and requests supported.
- **Telemetry throughput:** Number of execution events processed by the aggregation pipeline per second.
- **Metric accuracy:** Correctness of calculated throughput, latency percentiles, error rates, and response-code distributions.

12.2 Secondary Evaluation Metrics

- Controller-to-worker coordination latency.
- Worker failure recovery behavior.
- Real-time dashboard update latency.
- Storage scalability for historical test results.
- Accuracy of maximum sustainable throughput and breaking-point detection.
- Local-agent request forwarding latency and reliability.
- Scalability achieved by adding additional cloud workers.

13 Scalability *. . . horizontal growth*

The system is designed for horizontal scalability.

1. **Load-generator scalability:** Additional cloud workers can be added to increase traffic-generation capacity without requiring upgrades to the user's local machine.
2. **Remote testing of local applications:** A lightweight agent allows cloud workers to test applications bound to `localhost`, making the platform usable for developers and students with limited hardware.
3. **Aggregation scalability:** Telemetry processing can be distributed across multiple aggregation instances when event volume increases.
4. **Stateless services:** Controller and API services can be designed to remain stateless where possible, allowing multiple instances to run concurrently.
5. **Storage scalability:** Historical performance data can be indexed and partitioned according to test, timestamp, and application version.
6. **Asynchronous processing:** Streaming and asynchronous processing prevents high-volume telemetry from blocking request generation.

14 Project Scope and Deliverables *. . . iteration-friendly*

14.1 Initial Deliverable *. . . first iteration*

- Central controller for creating and managing tests.
- Single cloud load-generator worker capable of generating configurable concurrent HTTP requests.
- Basic local load-testing agent capable of establishing a secure connection to the platform and forwarding requests to a locally hosted application.
- Configurable virtual users, request rate, duration, ramp-up/ramp-down, headers, authentication, and payloads.
- Collection of request-level performance metrics.
- Basic metrics aggregation for throughput, latency, response codes, and errors.
- Persistent storage of test configurations and results.

14.2 Subsequent Deliverables

- Multiple distributed cloud load-generator workers.
- Local load-testing agent with secure connection management.
- Communication gateway for routing remote traffic to local applications.
- Streaming telemetry pipeline.
- Real-time WebSocket dashboard.
- Real-time performance graphs and visualization.

- P50/P95/P99 latency analysis.
- Adaptive stress testing and breaking-point detection.
- Configurable SLO-based performance evaluation.
- Historical performance comparison.
- Performance regression testing.
- CI/CD pipeline integration.
- Worker failure handling and fault tolerance.

15 Risks and Mitigations

- **Load-generator bottleneck:** Use asynchronous networking and distribute traffic across multiple cloud workers.
- **Local machine bottleneck:** Move request generation to remote workers so that the user's machine primarily processes the application under test rather than generating the complete workload.
- **Telemetry overload:** Use batching, asynchronous streaming, and aggregation instead of storing every event synchronously.
- **Incorrect percentile calculations:** Use validated percentile algorithms and test metric calculations against known datasets.
- **Worker failure:** Track worker heartbeats and isolate failed workers from the remaining test execution.
- **Secure local connectivity:** Authenticate local agents, use secure outbound connections, restrict request routing, and avoid exposing arbitrary local ports to remote users.
- **Local network bottleneck:** Clearly distinguish remote-to-local testing from cloud-to-cloud testing because bandwidth and network latency between the user's machine and the cloud can affect test results.
- **False breaking-point detection:** Evaluate multiple metrics and require configurable SLO violations rather than relying on a single latency measurement.
- **Storage growth:** Store aggregated metrics for long-term analysis while limiting unnecessary raw telemetry retention.

16 Summary

This project proposes a backend-focused distributed performance testing platform capable of generating large-scale HTTP workloads using a central controller and horizontally scalable cloud load-generator workers. A lightweight local agent will additionally allow users to test applications running on `localhost` without requiring their local machine to generate the complete workload. Unlike a basic load-testing tool that only generates requests and plots graphs, the proposed system focuses on distributed workload orchestration, secure remote-to-local request routing, asynchronous networking, telemetry streaming, real-time metric aggregation, performance visualization, adaptive stress testing, SLO-based capacity analysis, breaking-point detection, and performance regression testing. The visualization layer will transform aggregated

telemetry into real-time performance graphs, latency percentile trends, throughput and concurrency charts, response-code distributions, SLO indicators, capacity visualizations, and historical comparisons. The platform will provide a practical environment for studying and implementing backend engineering concepts including high concurrency, distributed systems, rate limiting, asynchronous processing, secure tunneling, worker coordination, fault tolerance, scheduling, real-time communication, performance benchmarking, CI/CD quality gates, performance visualization, and scalable data storage.